Project 1 - Fixed-Length Library Database

Create a Java application that manages a small library database using `RandomAccessFile`. The objective of the project is to practise working with fixed-length records and random access to a file. The database must store information about books, and each book must be represented by a fixed-length record in a binary file.

Each record must contain a book identifier, the title of the book, the number of available copies, and the price of the book in cents. The identifier must have a fixed maximum length of 12 characters and the title must have a fixed maximum length of 40 characters. The number of copies and the price must be stored as integers. You must decide how the information will be represented in the file and calculate the size of a complete record accordingly.

Create a class called `LibraryDB` that is responsible for managing the database file. The class must open a file using `RandomAccessFile` in read/write mode and must provide a method for adding a new book at the end of the file. When a text field is shorter than its maximum length, it must be padded so that every record has exactly the same size. When a text field is longer than its maximum length, it must be truncated.

The `LibraryDB` class must provide a method that returns the number of records currently stored in the database. It must also provide a method that receives a record number and returns the corresponding book. The method must use `seek()` to move directly to the appropriate position in the file rather than reading all previous records. Invalid record numbers must be detected and handled appropriately.

The class must also provide a method for updating an existing book. The method must receive the record number and the new information, move the file pointer to the beginning of that record, and overwrite the existing record with the new data. The implementation must preserve the fixed size of the records.

Create a separate class called `UseLibraryDB` that uses `LibraryDB`. This class must create or open a database file and populate it with at least five books when the database is empty. After populating the database, the program must display all the books stored in it. The information should be displayed in a clear table-like format rather than simply printing the Java objects using `toString()`.

The program must then update at least one existing book, for example by changing its number of available copies or its price, and display the records again so that the user can verify that the modification was successful.

The application must also display the number of records stored in the database. When displaying prices, the value stored in cents must be converted into a more readable format, such as euros with two decimal places.

Your implementation must separate the responsibilities of the database class and the application that uses it. `LibraryDB` should be responsible for accessing and modifying the file, while `UseLibraryDB` should be responsible for using the database and presenting information to the user.

As an additional requirement, your program must correctly close the database file when it finishes, including when an `IOException` occurs. The final program should demonstrate that records can be added, selected, updated, and displayed without having to read the entire file sequentially.

The project must be submitted before **October 28 at 11:59 p.m**. Any submission made after this deadline will be considered late.